scribs&pixels school · v1 · 2 October 2026

Victor's Task Program

Everything you, not Claude, have to do, written like code: explicit inputs, numbered steps, IF/ELSE branches, expected output and what to do on error. Work through it slowly, one procedure at a time.

00	How to run this program (read first)	DO NOW
01	answer_phase2_questions()	✓ DONE 3 Oct
02	confirm_school_rules()	✓ DONE 9 Oct
03	test_school_as_learner()	DO NOW · weekends
04	bring_back_ui_design()	WHEN READY
05	deploy_on_vercel()	✓ DONE 9 Oct
06	connect_subdomain()	✓ DONE 9 Oct
07	create_supabase_project()	✓ DONE 12 Oct
08	set_up_resend()	✓ DONE 15 Oct
09	deploy_email_nudges()	ON HOLD
10	switch_on_accounts()	NEXT
11	install_the_app()	WHEN READY
12	Understand what you did (05–11 explained)	READ
13	Appendix: words, tools and message templates	

Where things stand (15 Oct). Hosting, the subdomain, Supabase and Resend are done. Accounts are built into the app; **10 switches them on**. 09 (the nudge emails) comes after 10, once your own account has some progress in it.

00 · How to run this program

Read this page once. It works like the rules of a programming language: every procedure after it follows them.

The rules

- ☐ `01` **One procedure at a time.** Finish it, or stop it cleanly, before starting another. Never run two at once.
- ☐ `02` **Run the steps in order.** Don't skip a step because it "looks obvious". If a step says *copy*, copy; don't retype.
- ☐ `03` **Tick each** ☐ with a pen as you finish it. If you stop halfway, the last tick tells you where to resume.
- ☐ `04` **Check the EXPECTED OUTPUT** at the end of each procedure. If what you see matches, the procedure is done.
- ☐ `05` **ON ERROR, stop.** Don't improvise. Take a screenshot, then send Claude the error template from the Appendix (section 10).
- ☐ `06` **Secrets never go into chat, email, WhatsApp or the repo.** A secret is any value this document labels `SECRET`. Keep them in a password manager (Apple Passwords, 1Password or Bitwarden).

How to read a procedure

PROCEDURE	The name of the task, like a function name.
WHY	What it achieves, in one sentence.
INPUTS	What you need before you start. If you don't have every input, don't start.
TIME	Roughly how long it takes.
STEPS	Numbered actions, done in order.
IF / ELSE	A fork: do the branch that matches what you see.
EXPECTED OUTPUT	What "done" looks like.
ON ERROR	What to do when it doesn't work.

Notation

- **Bold text in steps** = a button, menu or label to click on screen, spelt exactly as it appears.
- `Grey monospace` = text to type or paste exactly, character for character.
- `<LIKE_THIS>` = a placeholder. Replace the whole thing, including the < >, with your own value.
- → = "then click". **Settings** → **Domains** means click Settings, then click Domains.
- "Terminal" = the Mac app called Terminal (press ⌘ **+ Space**, type `Terminal`, press Enter).

01 · answer_phase2_questions() DO NOW

`PROCEDURE answer_phase2_questions() → 3 answers`

WHY	Fullstack Weeks 11–14 can't be written until you decide these. Weeks 1–10 are written and live.
INPUTS	15 quiet minutes. Optional: the draft plan at `src/content/courses/fullstack-ml/phase-2-plan.md` (on GitHub: repo → that path).
TIME	15 minutes

- ☐ **01** **Question 1: order.** The draft is: Week 11 modern CSS (Grid, multi-page sites) → Weeks 12–13 React (components, then state and fetching) → Week 14 milestone in plain HTML/CSS/JS. Decide:
  IF you're happy with that order → answer `Order: keep`
  ELSE IF you want React earlier and the milestone built in React → answer `Order: React first`
- ☐ **02** **Question 2: milestone business.** Week 14's milestone is a multi-page website (Home · Menu · About · Contact) for a pretend Nigerian business. The suggestion is **"Mama Put Kitchen"**, a Lagos buka: menu with ₦ prices, opening hours, WhatsApp ordering. Alternatives: an Abuja tailor, an Enugu phone-repair shop.
  IF you like it → `Business: Mama Put Kitchen`
  ELSE → `Business: <YOUR_IDEA>` plus one sentence about what it sells.
- ☐ **03** **Question 3: Saturdays.** As drafted, the Saturdays of Weeks 11–14 all build towards the milestone site. The alternative is one separate project each Saturday.
  IF yes → `Saturdays: build the milestone` **ELSE** → `Saturdays: separate mini-projects`
- ☐ **04** Open Claude Code (Mac or claude.ai/code) in this project.
- ☐ **05** Paste this message, with your three answers filled in, and send it:

```
Phase 2 decisions:
Order: <keep | React first>
Business: <Mama Put Kitchen | your idea + one sentence>
Saturdays: <build the milestone | separate mini-projects>
Write Fullstack Weeks 11–14 when it's a weekday.
```

EXPECTED OUTPUT: Claude replies confirming the plan. On the next weekday it writes Weeks 11–14 and pushes them to the test site.

ON ERROR: If you can't decide, send `Phase 2: use your defaults`. The defaults are: keep the order, Mama Put Kitchen, Saturdays build the milestone.

02 · confirm_school_rules() DO NOW

```
PROCEDURE confirm_school_rules() → 5 decisions
```

WHY	The app runs on default rules that you haven't approved yet. Each rule lives in one place in the code, so changing it later is cheap, but approving them now stops guesswork.
INPUTS	The table below.
TIME	10 minutes

For each row, write **KEEP** or **CHANGE: <how>** in the last column.

Rule	Current default
R1 Pacing	"Today" = your first unfinished day. Missing an evening never puts you behind. ☐ KEEP ☐ CHANGE: _____
R2 Week quiz pass mark	The end-of-week quiz is self-marked; you pass when every question is answered and marked "got it". (Daily gates stay at 70%.) ☐ KEEP ☐ CHANGE: _____
R3 Streak	Counts Mon–Sat study days. Sunday is rest and never breaks or adds to it. ☐ KEEP ☐ CHANGE: _____
R4 "Need a nudge?" button	No behaviour yet. Ideas: send a reminder email in 1 hour · show the 10-minute version of today · open a motivational note. ☐ pick: _____
R5 Who writes new weeks	Claude writes them in Claude Code, a slice at a time, verified before pushing. ☐ KEEP ☐ CHANGE: _____

☐ **01** Fill in all five rows on paper.

☐ **02** Send Claude:

```
School rules:
R1 pacing: <KEEP | CHANGE: ...>
R2 week quiz: <KEEP | CHANGE: ...>
R3 streak: <KEEP | CHANGE: ...>
R4 nudge button: <your pick>
R5 writing weeks: <KEEP | CHANGE: ...>
```

EXPECTED OUTPUT: Claude moves these from "Open decisions" to "Decided" in CLAUDE.md, and changes code only for rows you marked CHANGE.

03 · test_school_as_learner() DO NOW · weekends

```
PROCEDURE test_school_as_learner(course) → bug reports
```

WHY	You're the first real learner. Things you trip over will trip up every learner after you.
INPUTS	Your phone **and** a laptop. The school: `school.scribsandpixels.com`
TIME	20–40 minutes per session. Repeat any weekend.

Know this first: progress is saved **in this browser only**. A different phone or browser, a private tab, or clearing site data starts you from zero. That's expected until accounts exist.
Checkpoints are strict: once you press **Start checkpoint**, leaving, reloading or closing the page counts as a failed attempt (60-minute cooldown). Start only when you have 10 calm minutes.

- ☐ **01** Open the test site on your phone.
- ☐ **02** Tap one course. Choose one per session: Fullstack, French or Excel.
- ☐ **03** Open the week you're actually on, then today's day.
- ☐ **04** Do the day **as a learner**: read, open the links, tick blocks, do the quiz, use any sandbox, write the log.
- ☐ **05** While doing it, note anything in these 5 bug types:
  - **BROKEN**: an error, a blank screen, a button that does nothing.
  - **WRONG**: a wrong quiz answer, a wrong number, a typo, a dead link.
  - **CONFUSING**: you didn't know what to do next.
  - **UGLY**: overlapping text, too small on the phone, bad colours.
  - **SLOW**: you waited more than 3 seconds.
- ☐ **06** For each bug: take a screenshot, then copy the address bar (the bit after `#/` says exactly which page).
- ☐ **07** Optional, to test checkpoints quickly: tap the round 🧪 button (bottom right of the screen) → **Force checkpoint on this day**. To skip a 60-minute cooldown: **End cooldown now**.
- ☐ **08** **IF** you want to start a course again → 🧪 → **Reset my progress** → **OK**. **ELSE** leave it.
- ☐ **09** Send all bugs from the session in one message, using the bug template in section 10.

EXPECTED OUTPUT: One message to Claude listing your bugs. Claude fixes them and replies with what changed.

ON ERROR: If the whole site is blank, reload once. If it's still blank, try another browser. If it's blank everywhere, send `Site is blank on <device/browser>` with a screenshot.

04 · bring_back_ui_design() WHEN READY

```
PROCEDURE bring_back_ui_design() → design handed to Claude Code
```

WHY	The test site has plain screens. The real look is being designed on claude.ai. Claude Code can only build it once the design is in the project.
INPUTS	Your finished design conversation or artifact on claude.ai.
TIME	15 minutes

- ☐ **01** On claude.ai, open the design conversation.
- ☐ **02** Ask it: `Give me the final design as one HTML file plus a list of every screen and what each button does.`
- ☐ **03** **IF** it produced an artifact → click it → **Share / Publish** (or the link icon) → copy the link. **ELSE** → copy the whole HTML it gave you into a file named `design.html`.
- ☐ **04** In Claude Code, send:

```
Here is the final UI design: <LINK or attach design.html>.
Screens and buttons: <paste the list>.
Build it, replacing the test screens. Keep all content, quizzes and sandboxes
working.
```

EXPECTED OUTPUT: Claude confirms it can read the design and plans the build. Building happens on weekdays.

ON ERROR: If Claude says it can't open the link, attach the HTML file instead (step 3, ELSE branch).

05 · deploy_on_vercel() ✓ DONE 9 Oct

```
PROCEDURE deploy_on_vercel() → a second live copy of the site
```

WHY	Vercel is faster and handles custom domains easily. GitHub Pages keeps working either way.
INPUTS	Your GitHub login (account **VictorAnimasahun**).
TIME	10 minutes

- ☐ **01** Go to `vercel.com` → **Sign Up** → **Continue with GitHub**. Choose the free **Hobby** plan.
- ☐ **02** On the dashboard: **Add New…** → **Project**.
- ☐ **03** Find `scribsandpixels-school` in the list → **Import**.
  IF it isn't listed → **Adjust GitHub App Permissions** → allow the repo → come back.
- ☐ **04** Check the settings. Change only what's different:
  - Framework Preset: `Vite`
  - Build Command: `npm run build`
  - Output Directory: `dist`
  - Environment Variables: **none**. Leave the section empty.
- ☐ **05** Click **Deploy**. Wait for the confetti (about 1–2 minutes).
- ☐ **06** Click the preview image. A new tab opens on a `….vercel.app` address.
- ☐ **07** Check that the home page shows the three courses, then open any day.
- ☐ **08** Send Claude: `Vercel is live at <URL>`

EXPECTED OUTPUT: The same school at a `.vercel.app` address. Every push to `main` redeploys it automatically.

ON ERROR: If the build fails, open the build log, scroll to the first red line, screenshot it and send it to Claude. Don't change settings by guessing.

06 · connect_subdomain() ✓ DONE 9 Oct

```
PROCEDURE connect_subdomain(domain) → school.<yourdomain>
```

WHY	A clean address like `school.scribsandpixels.com` instead of the long GitHub one.
INPUTS	(a) 05 done. (b) A login to the company that holds your domain (the "registrar" or DNS host, e.g. Namecheap, GoDaddy, Cloudflare, Whogohost). (c) The exact domain and the subdomain you want.
TIME	15 minutes, then up to 24 hours for the internet to catch up

- ☐ **01** Decide the address, e.g. `school.<yourdomain>`. Done: `school.scribsandpixels.com`
- ☐ **02** In Vercel: open the project → **Settings** → **Domains** → type the address → **Add**.
- ☐ **03** Vercel shows a record to create. It's usually: Type `CNAME`, Name `school`, Value `cname.vercel-dns.com`. Copy what *it* shows if that's different.
- ☐ **04** Log in to your registrar → find **DNS** / **DNS Records** / **Advanced DNS** for the domain.
- ☐ **05** **Add record** → fill in exactly the Type, Name and Value from step 3. Leave TTL on Automatic → **Save**.
  IF your registrar is Cloudflare → set the orange cloud (proxy) to **grey / DNS only**.
- ☐ **06** Back in Vercel → **Domains**. Wait until the status says **Valid Configuration** (a refresh every few minutes is fine).
- ☐ **07** Open the new address in your phone browser.
- ☐ **08** Send Claude: `Subdomain live: <address>. Make it the main host and update APP_URL references.`

EXPECTED OUTPUT: The school opens at your subdomain with a padlock (https).

ON ERROR: "Invalid configuration" after 24 hours → screenshot the registrar's DNS page and Vercel's Domains page, and send both. **Don't delete other DNS records**: they run your email and main website.

07 · create_supabase_project() ✓ DONE 12 Oct

`PROCEDURE create_supabase_project() → database with tables + 2 keys`

WHY	Supabase will store learners' accounts and progress, so progress follows you across devices. Only useful once Claude wires accounts into the app.
INPUTS	A GitHub login, a password manager, and the Mac with this project.
TIME	25 minutes

- ☐ **01** Go to `supabase.com` → **Start your project** → **Continue with GitHub**.
- ☐ **02** **New project**. Name: `scribsandpixels-school`. Region: **West EU (London)** or **Central EU (Frankfurt)**, the closest to Lagos. Plan: **Free**.
- ☐ **03** Database Password: click **Generate a password** → save it in your password manager as **SECRET** "Supabase DB password" → **Create new project**. Wait about 2 minutes.
- ☐ **04** Open the Terminal and check the Supabase tool runs. Nothing to install: `npx` (part of Node, already on the Mac) fetches it the first time, which takes about a minute.

  ```
  cd ~/Desktop/scribsandpixels-school
  npx supabase --version
  ```

  It prints a version number such as `2.120.0`. If it asks `Ok to proceed? (y)`, type `y` and press Enter.
  Every Supabase command below starts with `npx`. Don't use `brew install supabase`: on this Mac it fails with "Command Line Tools are too outdated".
- ☐ **05** Log in from the Terminal (a browser window opens; approve it):

  ```
  npx supabase login
  ```
- ☐ **06** In Supabase: **Project Settings** (gear icon) → **General** → copy **Project ID** (the "project ref", 20 letters).
- ☐ **07** In the Terminal, go to the project and link it (paste your ref; type the DB password when asked):

  ```
  cd ~/Desktop/scribsandpixels-school
  npx supabase link --project-ref <PROJECT_REF>
  ```
- ☐ **08** Create all the tables (this runs the migration file Claude wrote):

  ```
  npx supabase db push
  ```

  When asked `Do you want to push these migrations?` type `Y` and press Enter.
- ☐ **09** In Supabase: **Table Editor**. You should see tables including `profiles`, `enrollments` and `quiz_attempts`.

☐ **10** Get the two values. Both are safe for the browser.

- **Project URL**: already known, no need to look it up: `https://ookbfjjsnzgbrcgltkot.supabase.co` (it's `https://` + your project ref + `.supabase.co`).
- **Publishable key**: left sidebar → **Project Settings** (gear icon, bottom) → **API Keys**. Copy the key that starts with `sb_publishable_` (click the copy icon next to it).
  IF there's no `sb_publishable_` key → open the **Legacy API Keys** tab and copy **anon** (a long key starting with `eyJ`). Either works.
- **Never** copy `sb_secret_…` or `service_role`: those are secret.

☐ **11** In the Terminal, create the local settings file:

```
cp .env.example .env.local
open -e .env.local
```

Replace the two placeholder values with your URL and anon key → save (⌘S) → close.

☐ **12** Tell Supabase where the school lives, so login emails link to it (not to `localhost`): **Authentication** → **URL Configuration**.

- Site URL: `https://school.scribsandpixels.com` → **Save**
- Redirect URLs → **Add URL**, one at a time: `https://school.scribsandpixels.com/**`, `https://scribsandpixels-school.vercel.app/**`, `http://localhost:5173/**`

☐ **13** Give the live site the same two values: Vercel → the project → **Settings** → **Environment Variables**. Add `VITE_SUPABASE_URL` = your Project URL, and `VITE_SUPABASE_KEY` = the anon / publishable key (environments: all) → **Save**. Then **Deployments** → the top one → ⋯ → **Redeploy**.

☐ **14** Send Claude: `Supabase project created and migrated. .env.local is filled in.` (Don't paste any keys.)

NEVER copy the `service_role` key anywhere except the password manager. It bypasses all security. If it ever leaks: Supabase → API settings → roll (regenerate) it.

EXPECTED OUTPUT: The tables are visible in the Table Editor, and `.env.local` holds two real values.

ON ERROR: If `npx supabase db push` prints red text → copy the whole Terminal output (⌘A, ⌘C) → send it to Claude. Don't run it again repeatedly.

08 · set_up_resend() ✓ DONE 15 Oct

`PROCEDURE set_up_resend(domain) → verified sending domain + API key`

WHY	Resend sends the daily nudge and Sunday review emails, from an address on your own domain.
INPUTS	A registrar/DNS login (as in 06) and a password manager.
TIME	20 minutes, then up to a few hours for verification

- ☐ **01** Go to `resend.com` → **Sign up** (the free plan is enough).
- ☐ **02** **Domains** → **Add Domain** → type your domain, `scribsandpixels.com` → choose the region closest to you → **Add**.
- ☐ **03** Resend shows 3–4 DNS records (TXT, MX, DKIM). For **each** one: registrar → DNS → **Add record** → copy Type, Name and Value exactly → **Save**.
- ☐ **04** Back in Resend → **Verify DNS Records**. Repeat every 15 minutes until every row is green / **Verified**.
- ☐ **05** **API Keys** → **Create API Key**. Name: `school-nudges`. Permission: **Sending access** → **Add**.
- ☐ **06** Copy the key **now** (it starts with `re_` and is shown only once) → save it in your password manager as **`SECRET`** "Resend API key".
- ☐ **07** Decide the From address, e.g. `Scribs & Pixels <school@scribsandpixels.com>`. Write it here: _____
- ☐ **08** **`REQUIRED`** before anyone but you can sign in (Supabase's own email only reaches members of your Supabase team): Supabase → **Authentication** → **Emails** → **SMTP Settings** → enable custom SMTP: host `smtp.resend.com`, port `465`, username `resend`, password = your Resend API key, sender = your From address → **Save**.

EXPECTED OUTPUT: The domain shows **Verified** in Resend, and the API key is in your password manager.

ON ERROR: If records won't verify after 24 hours → screenshot Resend's domain page and your DNS page, and send both. A common cause is the registrar adding your domain to the Name automatically (e.g. `send.yourdomain.com.yourdomain.com`): type only the part Resend shows before your domain.

09 · deploy_email_nudges() ON HOLD

`PROCEDURE deploy_email_nudges() → hourly emails running`

WHY	Switches on the daily nudge (Mon–Sat, at each learner's chosen hour, only if they haven't studied) and the Sunday review digest.
INPUTS	07 and 08 done. Your project ref, Resend API key, From address, and the site address (APP_URL: `https://school.scribsandpixels.com`).
TIME	20 minutes

- ☐ **01** Make a cron password in the Terminal and save the output in your password manager as **SECRET** "CRON_SECRET":

```
openssl rand -hex 32
```

- ☐ **02** Set the four secrets. Paste this as **one** command after filling in the four placeholders; keep the quotes:

```
cd ~/Desktop/scribsandpixels-school
npx supabase secrets set RESEND_API_KEY="<re_...>" EMAIL_FROM="Scribs & Pixels
<school@scribsandpixels.com>" APP_URL="https://school.scribsandpixels.com"
CRON_SECRET="<from step 1>"
```

- ☐ **03** Upload the email function:

```
npx supabase functions deploy send-nudges --no-verify-jwt
```

- ☐ **04** Do a safe dry run, which shows what *would* be sent without sending anything (replace both placeholders):

```
curl -s -X POST "https://<PROJECT_REF>.supabase.co/functions/v1/send-nudges?
dry-run" -H "Authorization: Bearer <CRON_SECRET>"
```

  IF it prints JSON (even an empty list) → continue. **ELSE IF** it says `Unauthorized` → the CRON_SECRET doesn't match step 2; redo step 2.

- ☐ **05** In the project folder, open `supabase/setup/schedule-nudges.sql` in TextEdit. **Don't save changes to it**; you'll paste a filled-in copy instead.
- ☐ **06** Copy its whole content. In Supabase → **SQL Editor** → **New query** → paste.
- ☐ **07** In the pasted text only, replace `<project-ref>` with your project ref and `<CRON_SECRET>` with your cron secret.
- ☐ **08** Click **Run**.
- ☐ **09** Check it's scheduled: new query → `select jobname, schedule from cron.job;` → **Run**. You should see `send-nudges` with `0 * * * *`.
- ☐ **10** Send Claude: `Nudges deployed and scheduled. Dry run returned <paste the output>.`

EXPECTED OUTPUT: The cron job is listed and the dry run returns JSON. Real emails start once learners have accounts.

ON ERROR: If the SQL editor says `secret already exists`, you ran the script twice. Send Claude the message; don't try to delete vault secrets yourself.

10 · switch_on_accounts() NEXT

```
PROCEDURE switch_on_accounts() → you sign in, and your progress follows you between devices
```

WHY	The app can now save progress to your account. It needs two things only you can do: add the new tables to your database, and put the 6-digit code into the sign-in email.
INPUTS	07 and 08 done. The Mac with the project. Your Supabase database password (password manager) and your Mac login password (for the keychain prompt).
TIME	15 minutes

□ **01** Get the latest project and add the new tables (checkpoints, attempts, study days). Paste in the Terminal:

```
cd ~/Desktop/scribsandpixels-school
git pull
npx supabase db push
```

It lists `20261015120000_checkpoints_and_sync.sql` and asks `Do you want to push these migrations?` → type `Y` → Enter.
IF macOS asks for your **login keychain** password ("supabase wants to use… Supabase CLI") → type your Mac password → **Allow**. That's the Supabase login you saved in 07.
IF it asks for the database password → paste "Supabase DB password" from your password manager.

□ **02** Check it worked: Supabase → **Table Editor**. You should now also see `checkpoints`, `checkpoint_attempts` and `study_days`.

□ **03** Copy the email template's code to the clipboard (paste this in the Terminal; it prints nothing):

```
pbcopy < supabase/templates/sign-in.html
```

`pbcopy` puts a file's exact text on the clipboard. **Don't open the file in TextEdit**: TextEdit shows an HTML file as a formatted page, and copying that gives formatted text, not the code Supabase needs. In the template you'll see `{{ .Token }}` and `{{ .ConfirmationURL }}`: placeholders that Supabase swaps for the real code and link in each email it sends.

□ **04** Supabase → **Authentication** → **Emails** → **Templates** → **Magic Link**:

- Subject: `Your Scribs & Pixels sign-in code`
- Body (message): select everything in the box, delete it, paste (⌘V) → **Save**.

□ **05** Same page → **Confirm signup** → the same subject and the same pasted body → **Save**. (New learners get this one the first time; it must have the code too.)

□ **06** On the laptop: open `school.scribsandpixels.com` → **Account** → type your email → **Email me a sign-in code**.

- ☐ **07** Open the email (Gmail on your phone is fine). **IF** it shows a 6-digit code → type it on the laptop → **Sign in**. **ELSE** (only a link) → step 4 or 5 didn't save; redo them and ask for a new email.
- ☐ **08** You should see **Your account** and, a moment later, **Saved to your account**.
- ☐ **09** On your phone: `school.scribsandpixels.com` → **Account** → sign in the same way. The progress from the laptop appears.
- ☐ **10** Tick one block on the phone. On the laptop, open **Account** → **Download from my account now**. The tick appears there too.
- ☐ **11** Make yourself the school's admin. Supabase → **SQL Editor** → **New query** → paste:

  ```
  update public.profiles set role = 'admin' where email =
  'animasahunvictor1@gmail.com' returning email, role;
  ```

  → **Run**. The results show one row: your email and `admin`. (`returning` makes the editor show the row it changed.) **IF** it says `Success. No rows returned` → no account has that email yet; sign in first (step 6), then run it again.
  What this does: every account is a *student*. An *admin* can also **read** every learner's progress on the Learners page (ticks, finished days, streaks, checkpoints) but can't change anyone's data and can't read their daily logs. The app can't make anyone an admin (learners are only allowed to edit their own settings), so this is done here, as the database owner.
- ☐ **12** On the laptop: **Account** → **Download from my account now**. A **Learners** link appears (sidebar, and "See all learners" on the Account page). Open it: you see yourself.
- ☐ **13** Send Claude: `Accounts are on. Signed in on laptop and phone. Admin works.`

EXPECTED OUTPUT: Signed in on both devices, the same progress on both, and the email showing a code from `school@scribsandpixels.com`.

ON ERROR: If the Account page says *Couldn't save to your account: Could not find the table 'public.…' in the schema cache* → step 1 didn't finish; run `npx supabase db push` again. If the email lands in spam → mark it "Not spam" and tell Claude. Anything else → copy the exact message on screen and send it with the error template below.

11 · install_the_app() WHEN READY

`PROCEDURE install_the_app() → the school as an app on your phone`

WHY	An icon on your home screen, full screen, and lessons that open without data. Two ways: install the website (any phone, 1 minute) or the Android app file (APK).
INPUTS	Your phone. For the APK: an Android phone.
TIME	5 minutes

- ☐ **01** **Option A, any phone (recommended first):** open `school.scribsandpixels.com` in Chrome (Android) or Safari (iPhone).
  Android: Chrome menu ⋮ → **Add to Home screen** (or **Install app**) → **Install**.
  iPhone: the **Share** button → **Add to Home Screen** → **Add**.
- ☐ **02** Open it from the new icon. It opens full screen, without the browser bar. Open a few lessons once while online; after that they open offline too.
- ☐ **03** **Option B, the Android app (APK):** on the phone, open `github.com/VictorAnimasahun/scribsandpixels-school/releases/tag/android-latest` → tap `scribsandpixels-school.apk` to download it.
- ☐ **04** Open the downloaded file. **IF** Android says it can't install apps from this source → **Settings** → allow **Install unknown apps** for your browser → go back → **Install**.
- ☐ **05** **IF** Play Protect warns about an unknown developer → **More details** → **Install anyway**. It warns because the app isn't from the Play Store yet, not because something is wrong with it.
- ☐ **06** Open **Scribs & Pixels**. Sign in on the **Account** tab with the code from the email. The app can't use the link in the email: on a phone, links open the browser.
- ☐ **07** Later builds: download and install the new APK the same way. It installs over the old one and keeps your progress.
- ☐ **08** Send Claude: `App installed (A / B / both). Anything odd: <what>`

EXPECTED OUTPUT: The green "S" icon on your home screen; the school opens full screen; lessons you've opened work in airplane mode.

ON ERROR: "App not installed" when updating → an older build was signed with a different key: uninstall the app, then install the new one, and sign in again (your progress is in your account). The French "listen" and "speak" buttons don't work inside the Android app (its built-in browser has no speech); use the website in Chrome for those.

12 · Understand what you did

Every step in 05–11 is real engineering. This section explains each one: **what it is** (in plain words), **what each line means**, and **what breaks without it**. Read it slowly; this is Fullstack Weeks 15, 17, 19, 22 and 28 done for real.

1. Git, GitHub and "push to main"

What it is. Git keeps save points of the project, like the version history in Google Docs, but you decide when to save and you label each save. A *commit* is one labelled save. GitHub holds the shared copy online. *Pushing* uploads your new saves there. `main` is the official version of the school.

.github/workflows/pages.yml	A recipe GitHub runs on its own computers after every push to main: install the exact library versions (`npm ci`), run every test (`npm test`, `check:db`, `check:python`, `check:web`, `check:accounts`), build, then publish to GitHub Pages. If any step fails, Pages keeps the old version.
Vercel	Watches the same repo and builds every push to main by itself. **It doesn't wait for those tests**, so a push that breaks a test still reaches school.scribsandpixels.com. (On Claude's list to fix.)

> **Without it:** no history to go back to after a mistake, and no automatic publishing; someone would copy files to a server by hand.

2. Vercel: "Build Command `npm run build`, Output Directory `dist`"

What it is. The school is written as many source files. A *build* turns them into a few files a browser can load quickly, like a printer turning a manuscript into a printed book. Vercel runs the build on its computers and serves the result from servers around the world.

Framework Preset: Vite	Vite is the tool that does the building. The preset tells Vercel the usual settings for it.
Build Command	`npm run build` runs `tsc -b && vite build`: first check the code for type mistakes (`tsc`), then build it.
Output Directory: dist	Where the finished files land. Vercel publishes exactly that folder.
Why two hosts look different	GitHub Pages serves the school from a sub-folder (`/scribsandpixels-school/`), so its build is told that (`GITHUB_PAGES=true`). Vercel serves it from the root (`/`).

3. DNS: the internet's phonebook

What it is. Computers find each other by number (an IP address such as `216.198.79.1`); people use names. DNS is the phonebook that turns a name into a number. You bought `scribsandpixels.com` from Spaceship, and Spaceship also keeps its phonebook page (its *nameservers* are `launch1` / `launch2.spaceship.net`). Every row you added is one phonebook entry.

Host	The part before your domain. `school` means school.scribsandpixels.com. `@` means the bare domain itself.
Type A	"This name lives at this exact number." Your `@ → 216.198.79.1` is a Vercel server, which redirects scribsandpixels.com to www.

Type CNAME	"This name lives wherever *that other name* lives." `school → fb47b490b1a42d37.vercel-dns-017.com.` lets Vercel move its servers without you ever editing DNS again. The trailing dot means "this is the complete name; add nothing to it".
TTL 30 min	How long other computers may remember an answer before asking again. That's why a change can take up to that long to reach everyone.
HTTPS, the padlock	Vercel got a free certificate from **Let's Encrypt** proving the server really is school.scribsandpixels.com, and it encrypts the connection. It lasts 90 days (this one: 9 Oct 2026 – 7 Jan 2027) and Vercel renews it automatically. Typing `http://` gets a *308 redirect*: "permanently moved to the https address".

Without it: nobody can reach the school by name. Without HTTPS, browsers warn "Not secure" and anyone on the same Wi-Fi could read the traffic.

4. Supabase: the database and its keys

What it is. Supabase runs a **Postgres** database for you: tables of rows (like spreadsheets) with strict rules about what each row may contain, plus a web API in front of it and a login system.

Project ref `ookbfjjsnzgbrcgltkot`	The project's unique ID. The API address is built from it: `https://ookbfjjsnzgbrcgltkot.supabase.co`.
Database password	The master password of the database itself. Only the command-line tool uses it (to change the tables). The app never sees it.
`brew` vs `npx`	Homebrew is an app store for terminal tools; it builds some of them with Apple's Command Line Tools, which were out of date on your Mac, so it refused. `npx` runs a tool straight from npm (Node's package library) without installing it for good, and the Supabase tool is published there too. Same tool, different shop.
`npx supabase login`	Opens the browser and creates a *personal access token*: a key that lets the terminal tool act as you on supabase.com. It's stored in the macOS **Keychain** (the Mac's encrypted password vault). That's what the "supabase wants to use your confidential information" prompt was: the tool asking the vault for it.
`npx supabase link --project-ref …`	Tells the tool which project this folder belongs to, so later commands don't need the ID. It remembers it in `supabase/.temp`, which isn't uploaded to GitHub.
Migrations + `db push`	Database changes are files of SQL (the database's language), numbered by date in `supabase/migrations`, applied oldest first. `db push` sends only the ones the database hasn't had yet and records each one, so nothing runs twice. Changes to the database are then reviewed, versioned and repeatable, like code.
Row level security (RLS)	A bouncer on every row. Each request carries a signed note saying who is asking; a *policy* such as `user_id = auth.uid()` means "you may only see and change rows that are yours". Asked without logging in, every table answered with nothing: that was RLS working.

Publishable key `sb_publishable_…`	Identifies the app at the front door. Public by design: it's inside the website's code and anyone can read it. On its own it can do nothing the RLS rules don't allow.
Secret key / `service_role`	The master key: it skips RLS completely. Only for code that runs on a server (the nudge emails). Never in the app, never in chat.
`.env.local`	A settings file the app reads when it runs on your Mac. Git ignores it, so it never goes to GitHub.

Without RLS: anyone with the public key could read every learner's data. That's why it's the first thing the database tests attack.

5. Auth → URL Configuration

Site URL	Where login emails send people by default: `https://school.scribsandpixels.com`.
Redirect URLs	The only places a login may hand someone back to. `/**` means "any page on that site". `http://localhost:5173` is the school running on your own Mac (`npm run dev`).

Without it: a fake site could start a login and have it delivered to itself. The list is the guard against that. Missing entries make login links bounce to the wrong place (by default, `localhost`).

6. Vercel environment variables, and why "Config", not "Secret"

Names that start with `VITE_` are copied *into the website's code* during the build, so every visitor's browser receives them. That's fine for the URL and the publishable key (both public by design), and it's why marking them Secret would protect nothing. It's also why you clicked **Redeploy**: the values are baked in when the site is built, not read while it runs.

7. Resend and the email records

What it is. Sending email that actually reaches inboxes needs servers with a good reputation and proof that they're allowed to send for your domain. Resend runs the servers; your DNS records are the proof. Gmail checks them on every email.

DKIM · TXT `resend._domainkey` = `p=MIGf…`	A *public key*. Resend signs every email with the matching private key; Gmail looks up this record to check the signature. Like a wax seal: the record is the public picture of your seal. It proves the email was sent with your key and wasn't changed on the way.
SPF · CNAME `send` → `send.forge.rmta.net`	Every email carries a hidden return address for bounces (the *Return-Path*), here `@send.scribsandpixels.com`. Through this CNAME, Resend publishes the **guest list** of servers allowed to use it: `v=spf1 ip4:52.3.252.119 ip4:44.222.39.36 ip4:199.249.231.0/24 ~all` (Resend's own servers; `~all` means "treat anyone else as suspicious"), plus an MX pointing at `feedback.forge.rmta.net`, the mailbox where bounce and spam-complaint reports go, so Resend stops mailing dead addresses.

SPF · CNAME `rsend` → `rsend-euw1.forge.rmta.net`	The same job for a second route: Resend also sends through **Amazon SES in Ireland** (`eu-west-1`, the region you picked). Its guest list is `v=spf1 include:amazonses.com ~all` and its bounce mailbox is `feedback-smtp.eu-west-1.amazonses.com`. (Resend doesn't document this record; this is what your DNS actually returns.) Both are CNAMEs so Resend can update the server lists without asking you to edit DNS.
DMARC · TXT `_dmarc` = `v=DMARC1; p=none;`	Your published policy for mail that fails the checks above. `v=DMARC1` is the format version. `p=none` means "deliver it anyway; I'm only declaring a policy". Later it can become `p=quarantine` (spam folder) or `p=reject`. Since 2024, Gmail and Yahoo expect bulk senders to have one; without it, nudges drift to spam.
Spaceship's MX + `v=spf1 include:spf.efwd.spaceship.net`	The *receiving* side: mail sent *to* your domain is forwarded by Spaceship. Sending and receiving are separate, which is why you left these alone.
API key: Sending access, one domain	*Least privilege*: the key can send, only from scribsandpixels.com, and can't read or delete anything. If it ever leaked, the damage is limited and you'd roll (replace) it.
Supabase SMTP settings	SMTP is the language mail servers speak. Host `smtp.resend.com`; port `465` = encrypted from the first byte; username `resend` (always that word); password = the API key; sender `school@scribsandpixels.com`. Supabase now hands its login emails to Resend. Its own mailer is only for testing: it reaches your team members only, a few per hour.

8. How signing in works (code, link, PKCE)

You ask for an email; it holds a **6-digit code** and a **link**. The code works on any device: ask on the laptop, read it on your phone, type it on the laptop. The link uses **PKCE**: when you ask, your browser invents a secret and sends only its fingerprint; the link brings back a one-time code that only swaps for a login together with that original secret. So a link forwarded to, or stolen by, someone else is useless; that's also why a link opened on a different device says "use the code instead". A code or link works once and expires after an hour. *Signing out* removes your progress from that browser; it stays in your account.

9. The app on your phone (PWA, APK, signing)

Installable website (PWA)	A *manifest* file tells the phone the app's name, icon and colours, and to open full screen. A *service worker* is a small script the browser runs between the school and the internet, like an assistant who keeps copies of what you've read: when the internet is there it fetches the newest version; when it isn't, it hands you the saved copy. It never saves your account data.
Capacitor	Wraps the same school in a real Android app: a native shell around a built-in browser window that shows the school's files, which ship inside the app. One codebase for the website and the app.

APK	The file format of an Android app, like a `.exe` on Windows. The Play Store installs APKs for you; here you install it yourself, which is why Android asks you to allow "unknown apps" for your browser.
Signing key	Every APK is signed, like a wax seal from its maker. Android only lets a new version replace an installed one if the seals match, so nobody can push a fake update over your app. The test builds use one fixed test key, kept in the project, so each new build installs over the last. The Play Store will need a **private** release key: lose it and you can never update the app there again, so it goes in your password manager.
versionCode	A number that must go up with every version; Android won't install an older number over a newer one. GitHub's build counter provides it.
GitHub Actions + release	On every push, GitHub's computers build the APK (Java 21 and the Android SDK) and attach it to the `android-latest` release page, where you download it.

13 · Appendix: words, tools and message templates

Message template: error

```
ERROR in procedure <number + name>, step <number>.
What I did: <one sentence>
What I expected: <from EXPECTED OUTPUT>
What I saw: <exact text or "see screenshot">
Device/browser: <e.g. iPhone 13 Safari>
```

Message template: bugs from a test session

```
TEST SESSION <date>, course <Fullstack | French | Excel>, device <...>
1. [BROKEN|WRONG|CONFUSING|UGLY|SLOW] page #/<address after #/>
   What happened: ...
   What I expected: ...
2. ...
```

Words

Repo	The project's folder on GitHub: `github.com/VictorAnimasahun/scribsandpixels-school`. Claude pushes every finished change to its `main` branch.
Deploy	Putting the latest version on the internet. GitHub Pages does it automatically about 2 minutes after each push.
DNS / record	The internet's address book. A CNAME record says "this name points to that server".
Registrar	The company you bought the domain from. That's where DNS records are edited.
Migration	A file that creates database tables. Running it twice is safe for Supabase; it skips what's done.
Secret / key	A password for a service. "anon/publishable" keys are safe in the browser. "service_role", the Resend API key, CRON_SECRET and the DB password are **SECRET**.
Edge function	A small program that runs on Supabase's servers. Here: `send-nudges`, the email sender.
Cron	A timer. `0 * * * *` means "every hour, on the hour".
Dry run	A rehearsal that shows what would happen without doing it.

Where to find things

The school	`school.scribsandpixels.com` (main; Vercel). Copies: `scribsandpixels-school.vercel.app`, `victoranimasahun.github.io/scribsandpixels-school`
To-do list (Claude's)	`TODO.md` in the repo

Project rules	`CLAUDE.md` in the repo
Phase 2 draft	`src/content/courses/fullstack-ml/phase-2-plan.md`
This document's source	`docs/your-tasks/your-tasks.html`. Ask Claude to update it when tasks change.